Smart Greenhouse Monitoring & Control System

TLM Ledwaba (219119007) · PN Skosana (221720111)
Vaal University of Technology- Department of Computer Systems Engineering
Module: EIMSD4A-Microsystems Design | 16 May 2026

Abstract

Inside a greenhouse, temperature and humidity can shift within hours and when they do, crop damage follows quickly. For most smallholder farmers, however, the systems capable of catching those shifts in real time are simply too expensive to buy or too complex to run. This paper describes how we built a low-cost Smart greenhouse monitoring and control system using an ESP32-WROOM-32 microcontroller paired with a DHT22 sensor, specifically to close that gap.

The system works on two levels at once. On-site, a 16×2 I²C LCD screen and LED indicators give the farmer instant local feedback. Remotely, a live React web dashboard accessible from any browser shows the same data in real time. Every two seconds, the ESP32 reads temperature and humidity, checks configurable alert thresholds, drives the local outputs, and pushes a structured JSON payload to a locally hosted Mosquitto MQTT broker over Wi-Fi. The React dashboard picks up that data via WebSocket and, crucially, it can also send commands back to the hardware: silencing the buzzer, overriding LEDs, updating thresholds, and even pushing a custom message to the physical LCD, all without touching the firmware.

Testing confirmed that the end-to-end latency stays under five seconds, alerts fire within one sampling cycle, and RAM usage stays below 75 KB in line with benchmarks from Elnaggar et al. (2025). The design directly addresses three recurring problems in the existing literature: high system cost, interfaces that require technical expertise, and the absence of lightweight bidirectional IoT designs suited to farming environments with limited power and connectivity.

1. Introduction

Feeding the world is getting harder. As climate patterns become less predictable, growers have increasingly turned to greenhouses to shield their crops from the worst of those changes. But a greenhouse only protects what it monitors. Temperature and humidity are the two variables that matter most for plant health: even small, sustained deviations can slow photosynthesis, accelerate water loss, and open the door to disease.

The scale of what is at stake is well established. Rezaei et al. (2023) showed that wheat yields fall by 6.1% for every degree Celsius of warming under moderate climate scenarios. The same plant biology operates inside a greenhouse let the microclimate drift and losses follow regardless of the controlled environment around the plant

Most affordable monitoring systems share a critical blind spot: they deliver either on-site awareness or remote visibility, but not both (Singh et al., 2024). The commercial systems that do offer both tend to be expensive and hard to configure. Three problems repeat across the literature: the cost is prohibitive for small-scale farmers; the interfaces assume embedded systems knowledge that most farmers do not have; and there are very few lightweight, energy-efficient single-board designs that can operate reliably where power and internet infrastructure are limited.

This project was built to address all three of those gaps simultaneously. The result is an affordable single-board microsystem that provides simultaneous local and remote feedback with bidirectional remote control designed so that a farmer can operate it without any technical background. Four specific objectives guided the work:

- Design a hardware architecture around the ESP32-WROOM-32 microcontroller and DHT22 sensor.
- Implement a dual feedback mechanism: a local LCD and LED display combined with MQTT telemetry.
- Enable bidirectional dashboard-to-hardware control over MQTT.
- Validate the system against published performance benchmarks for latency, RAM usage, and alert response.

2. Literature Review

2.1 Environmental Monitoring in Greenhouse Agriculture

Getting temperature and humidity right is the foundation of greenhouse farming. Rezaei et al. (2023) conducted a large-scale analysis of climate change impacts on crop yields and found that even modest temperature increases measurably reduce output by disrupting enzyme function, accelerating water loss, and impairing photosynthesis. These same processes operate inside greenhouses, which means continuous environmental monitoring is not optional it is fundamental.

2.2 IoT Architectures for Agricultural Monitoring

Affordable microcontrollers, wireless protocols, and cloud platforms have made IoT-based agricultural monitoring more accessible than it was even a few years ago. Singh et al. (2024) built a greenhouse

automation system using the ESP32 paired with the ThingSpeak cloud platform. Their work is a useful reference point, but it also exposed the gap this project targets directly: their system provided only remote feedback, leaving farmers without any on-site awareness when connectivity dropped.

Helmy et al. (2023) validated MQTT as a transmission protocol across multiple ESP32 nodes hosted on a VPS broker, confirming reliable delivery to both local and remote interfaces. More recently, Rasyid et al. (2026) published a working ESP32-based greenhouse monitoring prototype, confirming this hardware-software combination is well-suited to agricultural IoT applications.

2.3 Performance Benchmarks and System Reliability

The clearest performance reference for a well-designed IoT monitoring system comes from Elnaggar et al. (2025): 99.8% MQTT message reliability, 99.2% uptime over 30 days, and alert latency consistently below five seconds. They also confirmed that an ESP32 running DHT22 sensors uses less than 75 KB of RAM under continuous operation. These figures serve as the performance benchmarks this project targets.

2.4 Gaps and Contribution of This Study

The literature consistently identifies three unresolved problems in affordable Smart greenhouse monitoring and control systems: high cost, interfaces too complex for non-technical users, and insufficient lightweight single-board designs. Critically, none of the reviewed systems offers bidirectional control the ability for a remote dashboard to actively modify hardware state in real time. This project contributes a solution that closes all four gaps simultaneously, using proven low-cost components in a novel architecture.

3. Methodology

3.1 Research Design

The project used a hardware-software co-design approach meaning the physical components and firmware were built and tested together rather than sequentially. Design requirements were drawn directly from the gaps in the literature and translated into functional specifications covering sensing accuracy, feedback types, communication protocol, remote control capability, and cloud integration. Implementation proceeded in four layers (sensing, local feedback, cloud telemetry, and bidirectional control), with each layer validated before the next was added.

3.2 Hardware Architecture and Components

The system is built around the following components. Each was selected for a specific role in the overall architecture:

Component	GPIO / Spec	Role in the System
ESP32-WROOM-32	Dual-core, Wi-Fi	Main microcontroller. Reads the DHT22, evaluates thresholds, drives local outputs, publishes MQTT data, and listens for dashboard commands.
DHT22 Sensor	±0.5°C / ±2% RH	Measures temperature and humidity every 2 seconds via a single-wire digital interface.
16×2 I²C LCD	GPIO21/22, 0x27	Displays live readings and system status on-site. Togglable between live and rolling average view.
Green LED	GPIO5	Lights up when conditions are within the safe range.
Red LED	GPIO2	Activates immediately when a threshold is breached.
Buzzer	GPIO23	Sounds an audio alert on breach; can be silenced remotely via the dashboard.
Push Button	GPIO18 (PULLUP)	Toggles the LCD display between live sensor readings and the 10-reading rolling average.
220Ω Resistors	In series with LEDs	Protect the ESP32 GPIO pins, which are rated to a maximum of 12 mA.

3.3 Communication Protocol and Control Architecture

MQTT was chosen as the communication protocol because of its low bandwidth consumption and well-documented reliability in constrained IoT deployments. A Mosquitto broker runs locally on the operator's PC, listening on port 1883 for standard MQTT connections and port 9001 for WebSocket connections from the React dashboard.

The ESP32 publishes structured JSON payloads to the topic greenhouse/data every two seconds and subscribes to greenhouse/control to receive commands from the dashboard. The React dashboard connects to Mosquitto via WebSocket and both receives incoming sensor data and sends control commands back. This bidirectional architecture absent from every system reviewed in the literature is the key architectural contribution of this work.

State Management for Threshold Inputs

During development, an important usability issue emerged: when a user typed a new threshold value into the dashboard input field, the value would "snap back" to the previous setting within seconds. Analysis

revealed the cause: each incoming MQTT message contained the ESP32's current threshold values, and the dashboard was overwriting its input state with every message.

To solve this, the dashboard implements a separated state pattern:
liveThrTemp / liveThrHum: Read-only values that update from every MQTT message, showing the ESP32's actual current thresholds

draftThrTemp / draftThrHum: Editable values that only change when the user types or clicks a preset button.When the user clicks the "Set" button, the dashboard publishes the draft value to the greenhouse/control topic. The ESP32 updates its internal threshold, and on the next sensor cycle, publishes the new value back. The dashboard then updates the liveThr display, confirming the change. This pattern ensures the input field never loses user input while still reflecting the true device state a recommended pattern for any IoT dashboard with bidirectional control.

Additionally, threshold presets ("Optimal" at 35°C/40% RH and "Trigger Alert" at 22°C/80% RH) allow one-click demonstration of system behaviour changes without manual data entry.

3.4 System Operation and Data Flow

The system runs a continuous monitoring cycle with real-time dashboard override capability. The sequence is as follows:

- The DHT22 polls temperature and humidity every 2 seconds. A 10-reading rolling average is computed continuously in parallel.
- Readings are displayed on the I²C LCD. Pressing the push button toggles between live and average mode.
- If any reading crosses the configured threshold (default: temperature above 32°C or humidity below 40% RH), the red LED activates and the buzzer sounds.
- A JSON payload is serialised and published to greenhouse/data via MQTT over Wi-Fi.
- The React dashboard renders live stat cards and a scrollable reading history table.
- From the dashboard, the operator can toggle the buzzer remotely, override LED states, update alert thresholds without reflashing firmware, and send a custom 16-character message to the physical LCD.
- Dashboard Interface Operation
- While the ESP32 handles local monitoring, the React dashboard provides remote visibility and control through a web browser. The dashboard connects to the Mosquitto broker via WebSocket

on port 9001 and subscribes to the greenhouse/data topic. Upon receiving each JSON payload, the dashboard performs four parallel updates:

- Stat Cards: Four live metric cards display current temperature, humidity, system status (Stable/ALERT), and push button state. Each card uses colour coding green for normal, red for alert conditions.
- Real-time Line Chart: A responsive line chart built with the Recharts library plots the last 20 temperature and humidity readings simultaneously. The chart updates every time a new MQTT message arrives, creating a live visual history of environmental trends.
- Scrollable History Table: Below the chart, a table maintains the same 20-reading buffer, showing timestamps, temperature, humidity, and alert status. Rows are colour-coded (red borders for alert states) and displayed in reverse chronological order, so the newest reading appears at the top.
- Hardware Control Panel: Six interactive controls allow remote manipulation of the ESP32 without firmware changes:
- Buzzer toggle: Override the buzzer manually or revert to auto mode
- LED override: Independently control red and green LEDs via checkboxes
- Alert thresholds: Update temperature maximum and humidity minimum values
- LCD messaging: Send a 16-character custom message to the physical LCD display
- All control commands are published to the greenhouse/control topic as JSON payloads. The ESP32 receives these commands within one second and executes the corresponding hardware action, then confirms the change by including the updated state in its next periodic data publication.

Offline Resilience: If the Wi-Fi connection drops, sensor monitoring, LED alerts, the buzzer, and the LCD display all continue operating without interruption. The remote dashboard is unavailable, but the farmer retains full on-site awareness.

3.5 Tools and Technologies

Firmware was developed in the Arduino IDE with the ESP32 board support package. Libraries used: PubSubClient (MQTT), ArduinoJson (payload serialisation), LiquidCrystal_I2C (LCD), and the standard DHT library. The React dashboard was scaffolded with Vite and uses the mqtt npm package for WebSocket connectivity. Mosquitto 2.x served as the local MQTT broker.

3.6 Dashboard User Interface Components

The React dashboard implements five distinct UI components, each serving a specific monitoring or control function:

3.6.1 Connection Status Indicator

Located in the header adjacent to the title, a dynamic status badge displays one of four states: "Connecting," "Connected," "Connection error," or "Disconnected." The badge uses colour coding (green for connected, red for errors) and includes a live status dot (●) for immediate visual recognition. This indicator updates in real time based on MQTT broker connectivity events, providing immediate feedback about system availability.

3.6.2 Scrollable History Table

Beneath the control panel, a table maintains the last 20 sensor readings in reverse chronological order (newest first). Each row displays:

Timestamp (HH:MM:SS format from the system clock)

Temperature in degrees Celsius

Humidity as percentage RH

Status badge ("ALERT" or "Stable")

Rows alternate between transparent and dark backgrounds for readability. When an alert condition occurs, the affected row's temperature and humidity values change to red, and the status badge switches to a red "ALERT" style. The table automatically scrolls as new data arrives, keeping the most recent reading visible without manual intervention.

3.6.3 Alert Banner

When the ESP32 reports an alert condition (temperature exceeding the maximum threshold or humidity falling below the minimum threshold), a prominent banner appears above all other content. The banner displays "SYSTEM ALERT, Temperature or humidity outside safe range" and includes a CSS pulse animation that continuously fades the banner in and out, ensuring the operator cannot miss critical alerts even when viewing other parts of the dashboard.

3.6.4 Control Buttons

All interactive controls use a reusable CtrlBtn component that accepts label, colour, and onClick handler props. Button colours follow semantic conventions:

Green (#16a34a): Positive actions (Turn ON, Stable states)

Red (#dc2626): Destructive or OFF actions

Blue (#2563eb): Apply/Override actions

Amber (#d97706): Configuration actions (Set thresholds)

Purple (#7c3aed): Communication actions (Send to LCD)

4. Results

The prototype met all core functional requirements during testing. The DHT22 delivered consistent readings within its specified accuracy limits at the 2-second sampling interval, and readings appeared on the 16×2 LCD without perceptible lag.

Alert behaviour was correct throughout: when a reading crossed the programmed threshold, the red LED activated within one sampling cycle (2 seconds) and the buzzer sounded. Both reverted automatically when conditions returned to the safe range.

MQTT publishing to the local Mosquitto broker was confirmed over Wi-Fi, with the React dashboard updating within the sub-five-second latency target. The WebSocket connection on port 9001 remained stable throughout testing. All four bidirectional control commands buzzer toggle, LED override, threshold update, and LCD messaging were confirmed functional, with the ESP32 responding to each command within one second of publication.

Rolling average computation over 10 readings functioned correctly, and the push button successfully toggled the LCD between live and average display modes. A seed value from the first reading prevents a blank display at startup.

Dashboard Visualization Performance

The React dashboard's real-time chart, implemented with the Recharts library, successfully rendered temperature and humidity trends simultaneously. With the history buffer configured to hold 20 readings (approximately 40 seconds of data at the 2-second sampling interval), chart updates occurred within 100 milliseconds of each MQTT message arrival. The dual-axis visualization proved particularly effective for demonstrating alert conditions: when a threshold breach occurred, both temperature and humidity lines displayed colour-coded warnings (red for alert, green for normal), allowing the operator to instantly correlate specific readings with system behaviour.

The scrollable history table, maintaining the same 20-reading buffer, rendered reverse-chronologically with alternating row colours for readability. Alert status badges (red "ALERT" vs green "Stable") updated synchronously with the chart. No rendering lag or UI blocking was observed during sustained operation, confirming that React's virtual DOM efficiently handled the 2-second update cadence.

RAM utilisation during continuous monitoring stayed below 75 KB, matching the benchmark reported by Elnaggar et al. (2025). Table 1 summarises all prototype test results.

Table 1: Summary of Prototype Test Results

Performance Metric	Target	Observed Result
DHT22 sampling interval	Configurable	2 seconds (default)
LCD update latency	No perceptible delay	No perceptible delay
MQTT dashboard latency	< 5 seconds	< 5 seconds
MQTT message reliability	99.8%	Pending 30-day evaluation
System uptime	99.2%	Pending 30-day evaluation
ESP32 RAM utilisation	< 75 KB	Sub-75 KB
LED alert response	Within 1 sampling cycle	Within 1 cycle (2s)
Buzzer remote control	< 1 second	< 1 second via MQTT
Dashboard control commands	4 commands	Buzzer, LED, threshold, LCD msg

5. Discussion

The prototype results show that this system achieves what the literature identified as missing. By combining the ESP32 and DHT22 with a local Mosquitto broker, a React dashboard, and a bidirectional MQTT architecture, the system gives farmers simultaneous on-site and remote awareness at a component cost that is accessible to small-scale operations. This directly addresses the core limitation of Singh et al. (2024), whose system provided only remote feedback and left farmers without on-site awareness when connectivity dropped.

Having both a local display and a remote dashboard is a practical necessity for the target environment, not just a convenience feature. In many smallholder farming areas, internet connectivity is intermittent. A remote-only system is therefore unreliable by design in exactly the contexts where reliability matters most. The LCD and LED indicators ensure the farmer retains actionable information even when Wi-Fi is unavailable, while the push button allows them to review rolling averages without any network dependency at all.

The bidirectional control architecture is this project's most significant departure from reviewed work. No reviewed system allowed a dashboard to modify hardware state at runtime without reflashing firmware. The ability to silence the buzzer, override LED states, update alert thresholds, and push messages to the physical LCD from any browser meaningfully increases practical usability in a real farming context. A

farmer who has adjusted a threshold to account for a seasonal crop change does not need a technician on-site to make that change, they can do it themselves.

The MQTT architecture follows the approach validated by Helmy et al. (2023). Using a locally hosted Mosquitto broker rather than a managed cloud service like Adafruit IO trades zero-configuration cloud hosting for full control over broker behaviour, no external dependencies, and no publish rate limits. The sub-five-second dashboard latency and sub-75 KB RAM usage are consistent with Elnaggar et al. (2025) benchmarks, providing early evidence that the system is on track to meet the 99.8% reliability and 99.2% uptime targets during the planned 30-day evaluation.

The main limitation is the system's dependence on local Wi-Fi for cloud telemetry and dashboard control. Where the network is unavailable, remote monitoring and control cease though local LCD and LED feedback continues uninterrupted. A future iteration could address this with a GSM/GPRS fallback module for remote connectivity, or SD card logging with batch upload when connectivity is restored.

6. Conclusion

This project set out to build a Smart greenhouse monitoring system and control Microsystem that is affordable, easy to use, and energy-efficient three qualities consistently absent from the reviewed solutions. The ESP32-WROOM-32 and DHT22-based system delivers real-time temperature and humidity monitoring with simultaneous on-site feedback (LCD and LED alerts) and remote visibility via a React dashboard connected through a locally hosted Mosquitto MQTT broker.

The key architectural contribution is bidirectional MQTT control: from a web browser, the operator can silence the buzzer, override LED states, update alert thresholds, and send custom messages to the physical LCD all without reflashing firmware. Prototype testing confirmed sub-five-second dashboard latency, correct LED and buzzer alert behaviour, rolling average computation, and RAM utilisation below 75 KB, all consistent with Elnaggar et al. (2025) benchmarks.

Two targets remain to be confirmed: 99.8% message reliability and 99.2% system uptime. These will be evaluated over a planned 30-day operational test. Future work should explore GSM fallback communication for off-grid environments, expanded sensor coverage (soil moisture, light intensity, CO_2), and automated actuator control for ventilation and irrigation.

References

Elnaggar, A. et al., 2025. Enhanced IoT-Based Environmental Monitoring System Using MQTT with Real-Time Alerts and Predictive Analytics. IAR Journal of Engineering and Technology, June 2025. [Online]. Available: https://iarconsortium.org/iarjet/191/2887/

Helmy, M. et al., 2023. Monitoring and Controlling of IoT-Based Greenhouse Parameters With the MQTT Protocol. Jurnal Nasional Teknik Elektro dan Teknologi Informasi, submitted August 2023, accepted December 2023. https://doi.org/10.25124/jnteti

Rasyid, A. N., Hamdani, D. and Setiawan, I., 2026. Design and Implementation of an IoT-Based Smart Greenhouse Monitoring System Prototype Using ESP32 Microcontroller. Jurnal Sains Student Research, 4(2), April 2026. [Online]. Available: https://ejurnal.kampusakademik.co.id/index.php/jssr/article/view/9577

Rezaei, E. E. et al., 2023. Climate change impacts on crop yields. Nature Reviews Earth & Environment, 4, pp. 831–846. https://doi.org/10.1038/s43017-023-00491-0

Singh, R., Hussain, A., Fezaa, L. H. A., Gupta, G., Singh, A. P. and Dogra, A., 2024. Greenhouse Automation System Using ESP32 and ThingSpeak for Temperature, Humidity, and Light Control. In: Advances in Data and Information Sciences, Lecture Notes in Networks and Systems, vol. 796. Singapore: Springer, pp. 431–440. https://doi.org/10.1007/978-981-99-6906-7_43